MLP-Based Power Prediction for CMOS Logic Gates

Inverter · 2-Input NAND · 2-Input NOR

Complete Google Colab Notebook — Run & Predict

■ QUICK START

1. Open Google Colab (colab.research.google.com)
2. Create a new notebook
3. Copy each CELL block from this PDF into separate Colab cells
4. Click **Runtime** → **Run all**
5. In the last cell, type your temperature and press Enter
6. All power values are printed instantly!

Dataset Overview (from Research Excel File):

Category	Gates	Input Combinations	Temp Range	Data Points
Total Avg Power	Inverter, NAND, NOR	— (averaged)	-40 to 120°C	10 per gate
Static Power	Inverter	0, 1	-40 to 120°C	10 × 2
Static Power	NAND	00, 01, 10, 11	-40 to 120°C	10 × 4
Static Power	NOR	00, 01, 10, 11	-40 to 120°C	10 × 4
Dynamic Power	Inverter, NAND, NOR	Worst-case derived	-40 to 120°C	10 per gate
Avg Dynamic Power	Inverter, NAND, NOR	Average derived	-40 to 120°C	10 per gate

1. MLP Architecture & Design Choices

This notebook trains **separate MLP (Multi-Layer Perceptron)** regression models for each power category and each gate/input-combination. This design ensures maximum accuracy for each individual output rather than using a single multi-output network.

Network Architecture (per model)

Layer	Neurons	Activation
Input	1 (Temperature °C)	—
Hidden 1	128	ReLU
Hidden 2	64	ReLU
Hidden 3	32	ReLU
Output	1 (Power in pW)	Linear

Training Configuration

Optimizer	Adam (Adaptive Moment Estimation)
Learning Rate	0.001
Max Iterations	10,000
Loss Function	Mean Squared Error (MSE)
Input Scaling	MinMax Normalization [0, 1]
Output Scaling	MinMax Normalization [0, 1] (per model)
Random Seed	42 (reproducible results)
Total Models Trained	17 separate MLP models

2. Complete Google Colab Code

■■ Copy each block below into a **separate cell** in Google Colab. Run them in order from Cell 1 to Cell 7.

CELL 1 — Install & Import Libraries

```
# Run this first — installs required packages
# (In Colab, scikit-learn, numpy, pandas, matplotlib are pre-installed)
# !pip install scikit-learn numpy pandas matplotlib -q   # uncomment if needed

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

print("All libraries imported successfully.")
```

CELL 2 — Dataset Definition

```
# Temperature values (degrees Celsius)
temperatures = np.array([-40, -20, 0, 20, 27, 40, 60, 80, 100, 120])

# TOTAL AVERAGE POWER (pW)
total_avg_power = {
    'Inverter': np.array([1712, 1750, 1790, 1835, 1849, 1882, 1927, 1974, 2024, 2075]),
    'NAND':     np.array([7525, 7720, 7921, 8109, 8183, 8309, 8502, 8696, 8888, 9084]),
    'NOR':      np.array([8214, 8510, 8800, 9098, 9209, 9403, 9686, 9997, 10290, 10580]),
}

# STATIC POWER - Inverter (input combinations: 0, 1)
static_inverter = {
    '0': np.array([3.847, 3.979, 4.297, 4.983, 5.359, 6.318, 8.702, 12.67, 18.89, 28.19]),
    '1': np.array([0.9941, 1.014, 1.195, 1.614, 5.359, 2.597, 4.650, 8.576, 15.48, 26.89]),
}

# STATIC POWER - NAND (input combinations: 00, 01, 10, 11)
static_nand = {
    '00': np.array([6.496, 6.527, 6.589, 6.805, 6.948, 7.332, 8.328, 10.04, 12.80, 17.11]),
    '01': np.array([3.993, 4.190, 4.681, 5.763, 6.363, 7.907, 11.97, 18.35, 28.74, 44.45]),
    '10': np.array([4.478, 4.695, 5.164, 6.156, 6.695, 8.081, 11.59, 17.43, 26.74, 40.87]),
    '11': np.array([1.988, 2.083, 2.391, 3.227, 3.745, 5.194, 9.313, 17.15, 30.95, 53.78]),
}

# STATIC POWER - NOR (input combinations: 00, 01, 10, 11)
static_nor = {
    '00': np.array([13.57, 13.85, 14.50, 15.89, 16.65, 18.57, 23.36, 31.30, 43.76, 62.38]),
    '01': np.array([6.634, 6.712, 6.938, 7.535, 7.905, 8.941, 11.91, 17.61, 27.76, 44.37]),
    '10': np.array([1.539, 1.639, 1.868, 2.428, 2.769, 3.724, 6.457, 11.73, 21.19, 37.34]),
    '11': np.array([0.7647, 0.7821, 0.8373, 0.9849, 1.088, 1.404, 2.372, 4.318, 7.290, 14.32]),
}

# DYNAMIC POWER = Total Avg - Static (Worst Case)
dynamic_power = {
    'Inverter': total_avg_power['Inverter'] - static_inverter['0'],
    'NAND':     total_avg_power['NAND']     - static_nand['00'],
    'NOR':      total_avg_power['NOR']      - static_nor['00'],
}

# AVERAGE DYNAMIC POWER
```

```
avg_dynamic_power = {
    'Inverter': total_avg_power['Inverter'] -
                (static_inverter['0'] + static_inverter['1']) / 2,
    'NAND':     total_avg_power['NAND'] -
                (static_nand['00']+static_nand['01']+static_nand['10']+static_nand['11'])/4,
    'NOR':      total_avg_power['NOR'] -
                (static_nor['00']+static_nor['01']+static_nor['10']+static_nor['11'])/4,
}

print("Dataset loaded. Temperature range:", temperatures[0], "to", temperatures[-1], "C")
```

CELL 3 — Build & Train MLP Models

```
X = temperatures.reshape(-1, 1)
scaler_X = MinMaxScaler()
X_scaled = scaler_X.fit_transform(X)

MLP_CONFIG = dict(
    hidden_layer_sizes=(128, 64, 32),
    activation='relu', solver='adam',
    max_iter=10000, random_state=42,
    learning_rate_init=0.001,
    n_iter_no_change=100,
)

models, scalers_y = {}, {}

def train_model(y_values):
    scaler_y = MinMaxScaler()
    y_scaled = scaler_y.fit_transform(y_values.reshape(-1,1)).ravel()
    model = MLPRegressor(**MLP_CONFIG)
    model.fit(X_scaled, y_scaled)
    return model, scaler_y

print("Training MLP models...")

models['total_avg'],   scalers_y['total_avg']   = {}, {}
models['static_inv'],  scalers_y['static_inv']  = {}, {}
models['static_nand'], scalers_y['static_nand'] = {}, {}
models['static_nor'],  scalers_y['static_nor']  = {}, {}
models['dynamic'],     scalers_y['dynamic']     = {}, {}
models['avg_dynamic'], scalers_y['avg_dynamic'] = {}, {}

for gate, vals in total_avg_power.items():
    models['total_avg'][gate], scalers_y['total_avg'][gate] = train_model(vals)

for c, vals in static_inverter.items():
    models['static_inv'][c], scalers_y['static_inv'][c] = train_model(vals)

for c, vals in static_nand.items():
    models['static_nand'][c], scalers_y['static_nand'][c] = train_model(vals)

for c, vals in static_nor.items():
    models['static_nor'][c], scalers_y['static_nor'][c] = train_model(vals)

for gate, vals in dynamic_power.items():
    models['dynamic'][gate], scalers_y['dynamic'][gate] = train_model(vals)

for gate, vals in avg_dynamic_power.items():
    models['avg_dynamic'][gate], scalers_y['avg_dynamic'][gate] = train_model(vals)

print("All 17 MLP models trained successfully!")
```

CELL 4 — Model Accuracy Evaluation

```
def evaluate(model, scaler_y, y_true):
    y_pred_s = model.predict(X_scaled)
    y_pred = scaler_y.inverse_transform(y_pred_s.reshape(-1,1)).ravel()
    return (r2_score(y_true, y_pred),
            mean_absolute_error(y_true, y_pred),
            np.sqrt(mean_squared_error(y_true, y_pred)))

print("MODEL ACCURACY SUMMARY")
print("=" * 60)

categories = [
    ("Total Average Power", 'total_avg', total_avg_power),
    ("Static Power - Inverter", 'static_inv', static_inverter),
    ("Static Power - NAND", 'static_nand', static_nand),
    ("Static Power - NOR", 'static_nor', static_nor),
    ("Dynamic Power", 'dynamic', dynamic_power),
    ("Average Dynamic Power", 'avg_dynamic', avg_dynamic_power),
]

for cat_name, key, data in categories:
    print(f"\n  {cat_name}")
    print("  " + "-" * 55)
    for sub_key, vals in data.items():
        r2, mae, rmse = evaluate(models[key][sub_key], scalers_y[key][sub_key], vals)
        print(f"  {sub_key:<12} | R2={r2:.6f} | MAE={mae:.4f} pW | RMSE={rmse:.4f} pW")

print("=" * 60)
```

CELL 5 — Prediction Function

```
def predict_all_powers(temp_celsius):
    T_scaled = scaler_X.transform([[temp_celsius]])

    def pred(model, scaler_y):
        val = model.predict(T_scaled)
        return float(scaler_y.inverse_transform(val.reshape(-1,1))[0][0])

    return {
        'temperature': temp_celsius,
        'total_avg_power': {
            'Inverter': pred(models['total_avg']['Inverter'], scalers_y['total_avg']['Inverter']),
            'NAND':     pred(models['total_avg']['NAND'],     scalers_y['total_avg']['NAND']),
            'NOR':      pred(models['total_avg']['NOR'],      scalers_y['total_avg']['NOR']),
        },
        'static_power': {
            'Inverter': {
                'input_0': pred(models['static_inv']['0'], scalers_y['static_inv']['0']),
                'input_1': pred(models['static_inv']['1'], scalers_y['static_inv']['1']),
            },
            'NAND': {
                'input_00': pred(models['static_nand']['00'], scalers_y['static_nand']['00']),
                'input_01': pred(models['static_nand']['01'], scalers_y['static_nand']['01']),
                'input_10': pred(models['static_nand']['10'], scalers_y['static_nand']['10']),
                'input_11': pred(models['static_nand']['11'], scalers_y['static_nand']['11']),
            },
            'NOR': {
                'input_00': pred(models['static_nor']['00'], scalers_y['static_nor']['00']),
                'input_01': pred(models['static_nor']['01'], scalers_y['static_nor']['01']),
                'input_10': pred(models['static_nor']['10'], scalers_y['static_nor']['10']),
                'input_11': pred(models['static_nor']['11'], scalers_y['static_nor']['11']),
            },
        },
        'dynamic_power': {
```

```
            'Inverter': pred(models['dynamic']['Inverter'], scalers_y['dynamic']['Inverter']),
            'NAND':     pred(models['dynamic']['NAND'],     scalers_y['dynamic']['NAND']),
            'NOR':      pred(models['dynamic']['NOR'],      scalers_y['dynamic']['NOR']),
        },
        'avg_dynamic_power': {
            'Inverter': pred(models['avg_dynamic']['Inverter'], scalers_y['avg_dynamic']['Inverter']),
            'NAND':     pred(models['avg_dynamic']['NAND'],     scalers_y['avg_dynamic']['NAND']),
            'NOR':      pred(models['avg_dynamic']['NOR'],      scalers_y['avg_dynamic']['NOR']),
        },
    }

def print_prediction(r):
    T = r['temperature']
    print("\n" + "="*65)
    print(f"  POWER PREDICTIONS AT T = {T} degrees C")
    print("="*65)
    print("\n  TOTAL AVERAGE POWER (pW)")
    for gate, val in r['total_avg_power'].items():
        print(f"     {gate:<12}: {val:.4f} pW")
    print("\n  STATIC POWER - WORST CASE (pW)")
    for gate, combos in r['static_power'].items():
        print(f"     {gate}:")
        for inp, val in combos.items():
            print(f"       {inp:<12}: {val:.4f} pW")
    print("\n  DYNAMIC POWER - WORST CASE (pW)")
    for gate, val in r['dynamic_power'].items():
        print(f"     {gate:<12}: {val:.4f} pW")
    print("\n  AVERAGE DYNAMIC POWER (pW)")
    for gate, val in r['avg_dynamic_power'].items():
        print(f"     {gate:<12}: {val:.4f} pW")
    print("="*65)

print("Prediction function ready.")
```

CELL 6 — Visualization (Actual vs Predicted)

```
T_fine = np.linspace(-40, 120, 500).reshape(-1, 1)
T_fine_scaled = scaler_X.transform(T_fine)

def get_curve(model, scaler_y):
    y_s = model.predict(T_fine_scaled)
    return scaler_y.inverse_transform(y_s.reshape(-1,1)).ravel()

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle('MLP Power Prediction - Actual vs Predicted', fontsize=15, fontweight='bold')
clr = {'Inverter': '#2196F3', 'NAND': '#FF5722', 'NOR': '#4CAF50'}
c4  = {'00':'#6A1B9A','01':'#AD1457','10':'#00695C','11':'#E65100'}
ci  = {'0':'#1565C0','1':'#EF6C00'}

def plot_gate(ax, title, model_key, data_dict, color_dict):
    for k, vals in data_dict.items():
        ax.scatter(temperatures, vals, color=color_dict[k], s=60, zorder=5, label=f'{k} actual')
        ax.plot(T_fine, get_curve(models[model_key][k], scalers_y[model_key][k]),
                color=color_dict[k], lw=2, ls='--', label=f'{k} MLP')
    ax.set_title(title); ax.set_xlabel('Temperature (C)')
    ax.set_ylabel('Power (pW)'); ax.legend(fontsize=7); ax.grid(alpha=0.3)

plot_gate(axes[0,0], 'Total Average Power',    'total_avg',   total_avg_power, clr)
plot_gate(axes[0,1], 'Static Power - Inverter','static_inv',  static_inverter, ci)
plot_gate(axes[0,2], 'Static Power - NAND',    'static_nand', static_nand,     c4)
plot_gate(axes[1,0], 'Static Power - NOR',     'static_nor',  static_nor,      c4)
plot_gate(axes[1,1], 'Dynamic Power (Worst)',  'dynamic',     dynamic_power,   clr)
plot_gate(axes[1,2], 'Average Dynamic Power',  'avg_dynamic', avg_dynamic_power, clr)

plt.tight_layout()
plt.savefig('mlp_power_curves.png', dpi=150, bbox_inches='tight')
plt.show()
print("Plot saved as mlp_power_curves.png")
```

CELL 7 — ■ User Input: Enter Temperature to Predict

```
# ============================================================
#  ENTER YOUR TEMPERATURE HERE
#  Range: -40 to 120 C (extrapolation beyond is possible)
# ============================================================

user_temp = float(input("Enter temperature (degrees C): "))
result = predict_all_powers(user_temp)
print_prediction(result)
```

■ **Optional — Batch Prediction:** Predict at multiple temperatures at once:

```
# OPTIONAL CELL: Batch predictions
batch_temps = [-30, 10, 50, 90, 110]
for t in batch_temps:
    result = predict_all_powers(t)
    print_prediction(result)
```

3. Expected Output Format

When you enter a temperature (e.g. 50°C), the notebook prints the following structured output:

```
================================================================
  POWER PREDICTIONS AT T = 50 degrees C
================================================================

  TOTAL AVERAGE POWER (pW)
     Inverter    : 1901.XXXX pW
     NAND        : 8393.XXXX pW
     NOR         : 9541.XXXX pW

  STATIC POWER - WORST CASE (pW)
     Inverter:
       input_0     : 7.XXXX pW
       input_1     : 3.XXXX pW
     NAND:
       input_00    : 7.XXXX pW
       input_01    : 9.XXXX pW
       input_10    : 9.XXXX pW
       input_11    : 6.XXXX pW
     NOR:
       input_00    : 20.XXXX pW
       input_01    : 9.XXXX pW
       input_10    : 4.XXXX pW
       input_11    : 1.XXXX pW

  DYNAMIC POWER - WORST CASE (pW)
     Inverter    : 1893.XXXX pW
     NAND        : 8385.XXXX pW
     NOR         : 9520.XXXX pW

  AVERAGE DYNAMIC POWER (pW)
     Inverter    : 1896.XXXX pW
     NAND        : 8374.XXXX pW
     NOR         : 9523.XXXX pW
================================================================
```

The model will also generate a **6-panel matplotlib figure** (saved as **mlp_power_curves.png**) showing Actual data points vs MLP predicted curves for all power categories.

4. Understanding the Dataset Structure

4.1 Power Categories Explained

Total Average Power	Average power drawn by the gate across switching activity. Includes both static and dynamic components. Unit: pW.
Static Power (Worst Case)	Leakage current power dissipated when the gate is not switching. Reported for EACH input logic combination (0/1 for Inverter, 00-11 for NAND/NOR). Worst case = highest leakage combination.
Dynamic Power (Worst Case)	Power dissipated during logic transitions. Derived as: Total Avg Power minus Static Worst Case. Represents switching energy.
Average Dynamic Power	Dynamic power averaged across all input combinations. More realistic operating estimate than worst case alone.

4.2 Input Combinations for Static Power

Static power depends on which logic values are at the gate inputs, because different combinations activate different transistors in the CMOS stack:

Gate	Input Combinations	Description
Inverter	0, 1	Single input — 2 combinations
2-input NAND	00, 01, 10, 11	Two inputs — 4 combinations
2-input NOR	00, 01, 10, 11	Two inputs — 4 combinations

4.3 Temperature vs Power Trend

All power metrics increase with temperature due to increased leakage current in CMOS transistors at higher temperatures. The relationship is **non-linear and exponential** at higher temperatures (above 80°C), which is precisely why an MLP neural network is well-suited — it learns this non-linear mapping from the data directly.

5. Complete Dataset Reference Table

Total Average Power (pW) and Static Power (pW) — All Gates, All Temperatures

Temp (°C)	INV Total	NAND Total	NOR Total	INV Static 0	INV Static 1
-40	1712	7525	8214	3.847	0.9941
-20	1750	7720	8510	3.979	1.014
0	1790	7921	8800	4.297	1.195
20	1835	8109	9098	4.983	1.614
27	1849	8183	9209	5.359	5.359
40	1882	8309	9403	6.318	2.597
60	1927	8502	9686	8.702	4.65
80	1974	8696	9997	12.67	8.576
100	2024	8888	10290	18.89	15.48
120	2075	9084	10580	28.19	26.89

Static Power — NAND Gate (all input combinations, pW)

Temp (°C)	NAND-00	NAND-01	NAND-10	NAND-11
-40	6.496	3.993	4.478	1.988
-20	6.527	4.19	4.695	2.083
0	6.589	4.681	5.164	2.391
20	6.805	5.763	6.156	3.227
27	6.948	6.363	6.695	3.745
40	7.332	7.907	8.081	5.194
60	8.328	11.97	11.59	9.313
80	10.04	18.35	17.43	17.15
100	12.8	28.74	26.74	30.95
120	17.11	44.45	40.87	53.78

Static Power — NOR Gate (all input combinations, pW)

Temp (°C)	NOR-00	NOR-01	NOR-10	NOR-11
-40	13.57	6.634	1.539	0.7647
-20	13.85	6.712	1.639	0.7821
0	14.5	6.938	1.868	0.8373
20	15.89	7.535	2.428	0.9849
27	16.65	7.905	2.769	1.088
40	18.57	8.941	3.724	1.404
60	23.36	11.91	6.457	2.372

80	31.3	17.61	11.73	4.318
100	43.76	27.76	21.19	7.29
120	62.38	44.37	37.34	14.32

6. Troubleshooting & Notes

• ModuleNotFoundError
→ Uncomment the !pip install line at the top of Cell 1 and run it.

• Predictions seem off for very high temperatures
→ The training data only goes to 120°C. Predictions beyond 120°C are extrapolations and may be less accurate. The MLP will still give a value, but treat it as an estimate.

• KeyboardInterrupt on input()
→ In some Colab environments you need to click on the cell and run it interactively. Alternatively, replace input() with a hardcoded value: user_temp = 75

• Different results on re-run
→ The random_state=42 seed ensures reproducibility. If you change this, results will vary slightly between runs.

• R2 slightly less than 1.0
→ With only 10 data points and an MLP, a tiny gap from perfect fit is expected. R2 > 0.999 means the model is practically perfect for research use.

This notebook was auto-generated from research dataset. MLP model trained using scikit-learn. All power values in picowatts (pW).